

Leitfaden Branches mit Git

Objektorientierte Softwareentwicklung

4 August, 2026

Table of Contents

1	Ziel des Dokumentes	4
2	Definitionen	5
2.1	main-Branch	5
2.2	Feature-Branches	5
2.3	Pull Request.....	5
2.4	Release-Tags	5
2.5	Release-Branches.....	6
3	Namenskonventionen & Beschränkungen	7
4	Projektspezifische Erweiterungen	8
4.1	Eigene Branchtypen	8
4.2	Eigene Tags	8
4.3	Eigene Regeln für Pull Requests	8
5	Präsentation	9
6	Änderungshistorie.....	10

Inhalt

[Ziel des Dokumentes \(see page 4\)](#)

[Definitionen \(see page 5\)](#)

[main-Branch \(see page 5\)](#)

[Feature-Branches \(see page 5\)](#)

[Pull Request \(see page 5\)](#)

[Release-Tags \(see page 5\)](#)

[Release-Branches \(see page 6\)](#)

[Namenskonventionen & Beschränkungen \(see page 7\)](#)

[Projektspezifische Erweiterungen \(see page 8\)](#)

[Eigene Branchtypen \(see page 8\)](#)

[Eigene Tags \(see page 8\)](#)

[Eigene Regeln für Pull Requests \(see page 8\)](#)

[Präsentation \(see page 9\)](#)

[Änderungshistorie \(see page 10\)](#)

1 Ziel des Dokumentes

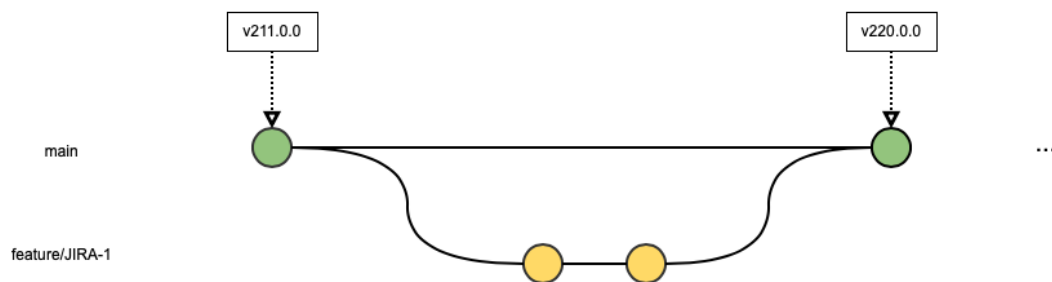
Die Erstellung und das spätere Mergen von Branches ist eine grundlegende Funktionalität von Git und sollte von jedem Entwickler verstanden werden. Außerdem sind Branches die Voraussetzung für die Nutzung von Pull Requests. Um ein einheitliches Verständnis der Branches innerhalb der F-I zu gewährleisten, werden in diesem Dokument grundlegende Namenskonventionen vorgestellt. Die tatsächliche Ausgestaltung der Branches erfolgt dann in den Teams bzw. den Plattform-Teams selbst.

[\(nach oben\)](#) [\(see page 3\)](#)

2 Definitionen

2.1 main-Branch

Der main-Branch ist der einzige verpflichtende, dauerhafte Branch. In diesem Branch kann der Entwicklungsfortschritt betrachtet werden und Code, der nach Produktion gebracht wird, findet sich grundsätzlich in diesem Branch wieder. Der Branch wird in GitHub deshalb besonders geschützt, sodass jede Änderung einen zweiten Entwickler benötigt.



2.2 Feature-Branches

Wenn neue Funktionalität entwickelt wird, erstellt sich der Entwickler einen Feature-Branch (z.B. feature/x) mit dem Stand aus main. Auf diesem Feature-Branch findet nun die Entwicklung eines möglichst kleinen, aber abgeschlossenen Features statt.

2.3 Pull Request

Sobald die Entwicklung der Funktionalität abgeschlossen ist und der Feature-Branch stabil ist, stellt der Entwickler einen Pull Request "feature/x -> main", den ein anderer Entwickler begutachten soll. Dieser zweite Entwickler führt dann ein kurzes Review durch und stellt z.B. sicher, dass der Build erfolgreich ist. Nach erfolgreichem Review kann der Pull Request von einem [Maintainer](#)¹ (dies kann einer der beiden Entwickler sein) gemergt und der Feature-Branch gelöscht werden.

Hinweis: Eine Prüfung auf Schadcode ist Mindestanforderung im Review von Pull Requests (siehe [Leitfaden Anwendungs-Codereview \(Diff\)](#)²).

2.4 Release-Tags

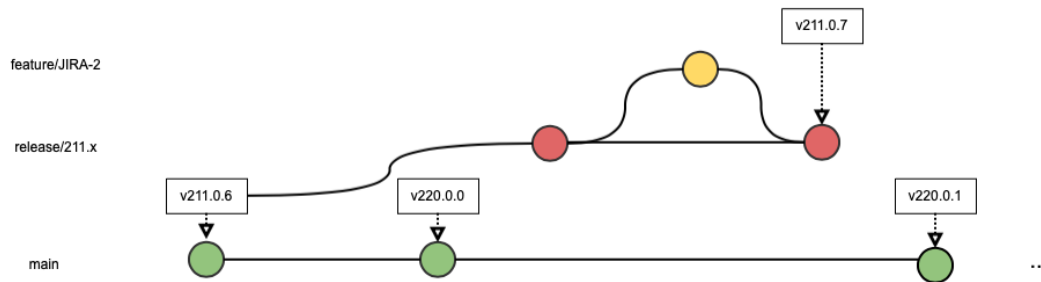
Wenn der Stand auf der main-Branch einen releasefähigen Stand hat, kann ein Entwickler ein Release-Tag erzeugen. Damit wird der Zustand unter einem Tag mit einer eindeutigen Releasenummer festgehalten. Sobald für ein Release keine Änderungen mehr erwartet werden, kann jetzt mit der Entwicklung für das Folge-Release auf dem main-Branch begonnen werden.

1. <https://wiki.intern/pages/viewpage.action?pageId=444323573>

2. <https://wiki.intern/spaces/OOP/pages/126255594/Leitfaden+Anwendungs-Codereview+Diff>

2.5 Release-Branches

Falls parallel zum Release auf main noch weitere Releases entwickelt werden, werden diese auf Release-Branches entwickelt – beispielsweise für Releases, die sich noch in Wartung befinden. Die Release-Branches sollten gelöscht werden, wenn keine Features mehr für das Release erwartet werden, da sie bei Bedarf aus den Release-Tags wiederhergestellt werden können.



[\(nach oben\)](#) (see page 3)

3 Namenskonventionen & Beschränkungen

Typ	Benennung	Beispiele	Beschränkungen
main-Branch (geschützt)	main	main	Änderung nur über Pull Requests im 4-Augenprinzip (GitHub: "Branch Protection").
Release-Branch (geschützt)	release / {Release-Version} WARTUNG - {Release-Version} WART - {Release-Version}	release/260 WARTUNG-260 WART-260	Änderung nur über Pull Requests im 4-Augenprinzip (GitHub: "Branch Protection"). Release-Branches dürfen nur auf Basis von Commits einer anderen geschützten Branch oder Release-Tags erstellt werden. Hinweis: Die Release-Version darf kein "/" enthalten
Feature-Branch	feature / {Feature-Name}	feature/JIRA-123, feature/mkgLogin	keine
Release-Tag (geschützt)	v {Release-Version}	v251.0.0, v251.0.1, v251.0.2, v260.0.0, v260.0.1	Release-Tags dürfen nur auf Basis von Commits erstellt werden, die auf einer geschützten Branch liegen. Release-Tags dürfen nach der Erstellung nicht mehr gelöscht werden. (GitHub: "Tag Protection"). Hinweis: Die Release-Version darf kein "/" enthalten

Im Idealfall werden die Release-Versionen für die Release-Tags und die Release-Branches nach dem [Semver](https://semver.org)³-Standard (MAJOR.MINOR.PATCH) vergeben. So könnten z.B. die Versionen v250.0.0, v250.0.1 und v250.0.2 auf dem Release-Branch release/250.0.x gebaut werden. Die Vorgabe der Release-Versionen in den Release-Branches und Release-Tags erfolgt durch die Teams bzw. Plattform-Teams und wird nicht in diesem Dokument geregelt.

(nach oben) (see page 3)

3. <https://semver.org>

4 Projektspezifische Erweiterungen

4.1 Eigene Branchtypen

Falls ein Projekt spezielle Anforderungen hat, können eigene Branch-Typen definiert werden. Zum Beispiel können epic-Branches zur Zusammenfassung von mehreren Features bei einer größeren Änderung (z.B. Java-Upgrade) verwendet werden.

4.2 Eigene Tags

Bei Bedarf können weitere Tags ohne v-Präfix verwendet werden, um Code-Stände zu markieren. Diese dürfen auch wieder gelöscht werden.

4.3 Eigene Regeln für Pull Requests

Um eine einheitliche Behandlung von Pull Requests im Entwicklungsteam zu gewährleisten, sollte im Team definiert werden, welche Prüfungen (z.B. ein erfolgreicher Build, Test oder die Erwähnung eines JIRA-Vorgangs) beim Review von Pull Requests überprüft werden sollen. Möglicher Name: "Definition of Done".

[\(nach oben\)](#) [\(see page 3\)](#)

5 Präsentation



[\(nach oben\)](#) [\(see page 3\)](#)

6 Änderungshistorie

In früheren Versionen dieses Leitfadens (vor Januar 2023) wurden Support-Banches und Release-Banches mit einer anderen Definition verwendet. Diese wurde jetzt zusammengefasst und dadurch vereinfacht. Support-Banches sollten somit jetzt mit "release/" benannt werden.

[\(nach oben\)](#) [\(see page 3\)](#)